



A K T U

B.Tech III-Year

CS IT & CS Allied

5th Semester



DBMS: Database Management System

ONE SHOT Revision

Unit-2

Relational data Model and Language

AKTU

DBMS-Database Management System

Unit-2 Syllabus

Relational data Model and Language: Relational Data Model Concepts, Integrity Constraints, Entity Integrity, Referential Integrity, Keys Constraints, Domain Constraints, Relational Algebra, Relational Calculus, Tuple and Domain Calculus. Introduction on SQL: Characteristics of SQL, Advantage of SQL. SQL Data Type and Literals. Types of SQL Commands. SQL Operators and Their Procedure. Tables, Views and Indexes. Queries and Sub Queries. Aggregate Functions. Insert, Update and Delete Operations, Joins, Unions, Intersection, Minus, Cursors, Triggers, Procedures in SQL/PL SQL

Unit-2 AKTU PYQs

Q.1.	Any numerical on relational algebra	AKTU 2020-21,2021-22,2022-23,2023-24
Q.2	What is relational algebra explain in detail	AKTU 2015-16,2017-18,2018-19
Q.3	Explain truncate ,drop and delete command	AKTU 2024-25
Q.4	Explain triggers and its types	AKTU 2024-25
Q.5	Explain join? Discuss all joins with example	AKTU 2024-25
Q.6	Explain equijoin in database.	AKTU 2021-22
Q.7	Explain the clause term in SQL	AKTU 2021-22
Q.8	Which Relational algebra operation require the participating table to be union compatible? Give the reason	AKTU 2021-22
Q.9	What is DDL and DML and also explain the feature of SQL	AKTU 2020-21
Q.10	Explain different integrity constraint.	AKTU 2020-21/18-19
Q.11	Write SQL query for different aggregate function.	AKTU 2020-21
Q.12	Explain tuple calculus and domain relation calculus	AKTU 2018-19

Relational Algebra

- Relational algebra is a procedural query language in DBMS used to retrieve data from relational databases. It works on relations (tables) as input and produces another relation (table) as output.

Relational algebra broadly divided into -

1. Fundamental Operations of Relational

Algebra:

1.1 Selection (σ)

1.2 Projection (π)

1.3 Union ($\cup$)

1.4 Set Difference ($-$)

1.5 Cartesian Product ($\times$)

1.6 Rename (ρ)

2. Advanced (Derived) Operations

2.1 Intersection ($\cap$)

2.2 Join ($\bowtie$) $\rightarrow$ (Theta, Equi, Natural, Outer, inner)

2.3 Division ($\div$)

2.4 Assignment ($\leftarrow$)

1.Fundamental Operations of Relational Algebra:

1.1.Selection (σ)

- Selection picks rows (tuples) from a relation (table) that satisfy a given condition.
- **Syntax:** $\sigma_{\langle \text{condition} \rangle}(\text{Relation})$
- **Output:** A new relation with only those rows that match the condition.
- **Works on:** Rows (not columns).

Table : Student

RollNo	Name	Dept	Year
101	Aditi	CSE	2023
102	Rohan	ECE	2024
103	Meena	CSE	2023
104	Aman	ME	2025
105	Priya	CSE	2024

a. Find all CSE students

$\sigma_{\text{Dept} = \text{'CSE'}}(\text{STUDENT})$

b. Find students of Year 2023

$\sigma_{\text{Year} = 2023}(\text{STUDENT})$

c. Find CSE students of Year 2023

$\sigma_{(\text{Dept} = \text{'CSE'} \wedge \text{Year} = 2023)}(\text{STUDENT})$

Solution Table c:			
RollNo	Name	Dept	Year
101	Aditi	CSE	2023
103	Meena	CSE	2023

Solution Table a:			
RollNo	Name	Dept	Year
101	Aditi	CSE	2023
103	Meena	CSE	2023
105	Priya	CSE	2024

Solution Table b:			
RollNo	Name	Dept	Year
101	Aditi	CSE	2023
103	Meena	CSE	2023

1.2. Projection (π)

- Projection picks columns (attributes) from a relation (table).
- **Syntax:** $\pi_{\langle \text{attribute-list} \rangle}(\text{Relation})$
- **Output:** A new relation containing only the specified columns.
- **Works on:** Columns (not rows).
- Projection automatically removes duplicates
- **Table : STUDENT**

RollNo	Name	Dept	Year
101	Aditi	CSE	2023
102	Rohan	ECE	2024
103	Meena	CSE	2023
104	Aman	ME	2025
105	Priya	CSE	2024

a. Get only student names

$\pi_{\text{Name}}(\text{STUDENT})$

b. Get names and departments

$\pi_{\text{Name, Dept}}(\text{STUDENT})$

c. Get unique departments

$\pi_{\text{Dept}}(\text{STUDENT})$

Solution Table a:
Name
Aditi
Rohan
Meena
Aman
Priya

Solution Table b:	
Name	Dept
Aditi	CSE
Rohan	ECE
Meena	CSE
Aman	ME
Priya	CSE

Solution Table c:
Dept
CSE
ECE
ME

1.3. Union (U)

Union combines tuples (rows) from two relations and removes duplicates.

- **Syntax:** $R \cup S$
- **Output:** A new relation containing all tuples from both relations.
- **Conditions for Union:** (Union Compatibility)
 - Two relations must be union compatible:
 - Same number of attributes (columns).
 - Same domain (data type) for corresponding attributes.

Relation R: STUDENT1	
RollNo	Name
101	Aditi
102	Rohan
103	Meena

Relation S: STUDENT2	
RollNo	Name
103	Meena
104	Aman
105	Priya

STUDENT1 U STUDENT2	
RollNo	Name
101	Aditi
102	Rohan
103	Meena
104	Aman
105	Priya

1.4.Set Difference (–)

The set difference operation returns tuples that are in the first relation but not in the second.

- **Syntax:** $R - S$
- **Output:** A new relation containing tuples from R that are not present in S

Conditions for Set Difference

- Relations R and S must be union compatible:
 - Same number of attributes (columns).
 - Same domain (data type) for corresponding attributes

Relation R: STUDENT1	
RollNo	Name
101	Aditi
102	Rohan
103	Meena

Relation S STUDENT 2	
RollNo	Name
103	Meena
104	Aman
105	Priya

Note:

- Not commutative $\rightarrow R - S \neq S - R$.
- Removes duplicates automatically.

STUDENT1 – STUDENT2	
RollNo	Name
101	Aditi
102	Rohan

STUDENT2 – STUDENT1	
RollNo	Name
104	Aman
105	Priya

1.5.Cartesian Product ($\times$)

- Cartesian Product pairs every tuple of one relation with every tuple of another relation.
- **Syntax:** $R \times S$
- **Output:** A new relation with all possible combinations of tuples

Relation S: COURSE	
CourseID	CourseName
C1	DBMS
C2	OS

Relation R: STUDENT	
RollNo	Name
101	Aditi
102	Rohan

STUDENT $\times$ COURSE			
Roll No	Name	CourseID	CourseName
101	Aditi	C1	DBMS
101	Aditi	C2	OS
102	Rohan	C1	DBMS
102	Rohan	C2	OS

1.6. Rename (ρ)

- The Rename operation is used to rename a relation (table) or its attributes (columns).
- **Syntax:** ρ NewName(Relation) or ρ (NewName(AttributeList))(Relation)
- **Output:** A relation with the same data, but with new names.

Why Rename is Needed?

- To avoid confusion when the same relation is used more than once (especially in Cartesian Product and Join).
- To give more meaningful names to attributes.

Original Relation: STUDENT		
RollNo	Name	Dept
101	Aditi	CSE
102	Rohan	ECE

Query: ρ Learner (STUDENT)		
Learner		
RollNo	Name	Dept
101	Aditi	CSE
102	Rohan	ECE

Original Relation: STUDENT		
RollNo	Name	Dept
101	Aditi	CSE
102	Rohan	ECE

Query: ρ (Learner(RNo, StudentName, Department)) (STUDENT)		
Learner		
RNo	StudentName	Department
101	Aditi	CSE
102	Rohan	ECE

2. Advanced (Derived) Operations

2.1. Intersection ($\cap$)

- Intersection returns the common tuples that appear in both relations.
- **Symbol:** $R \cap S$
- **Output:** A new relation with only the tuples present in both R and S

Conditions for Intersection

- Both relations must be union compatible:
 - Same number of attributes (columns).
 - Same domain (data type) for corresponding attributes.

Note:

- If no common tuples → result is empty relation ($\emptyset$).
- Intersection can be expressed using Set Difference:

$$R \cap S = R - (R - S)$$

Relation R: STUDENT1	
RollNo	Name
101	Aditi
102	Rohan
103	Meena

Relation S: STUDENT2	
RollNo	Name
103	Meena
104	Aman
105	Priya

OUTPUT	
RollNo	Name
103	Meena

It is a relational algebra operation that combines two relations (tables) based on a related attribute.

➤ It is basically a Cartesian Product ($\times$) followed by a Selection (σ).

➤ **Purpose:** To link information stored in different relations.

General Form:

$$R \bowtie_{\text{condition}} S = \sigma_{\text{condition}} (R \times S)$$

Relation: STUDENT		
SID	Name	Dept
1	Aditi	CSE
2	Rohan	ECE
3	Meena	CSE

Relation: DEPARTMENT	
Dept	HOD
CSE	Dr. Rao
ECE	Dr. Sen
ME	Dr. Roy

STUDENT $\bowtie_{\text{STUDENT . Dept = DEPARTMENT . Dept}}$ DEPARTMENT				
SID	Name	Student .Dept	Depart ment.D ept	HOD
1	Aditi	CSE	CSE	Dr. Rao
2	Rohan	ECE	ECE	Dr. Sen
3	Meena	CSE	CSE	Dr. Rao

Types of join

a. Inner join

1.1 Theta Join

1.2. Equi Join

1.3. Natural Join

b.Outer join

2.1.Left outer join

2.2.Right outer join

3.3.Full outer join

3. Other join (semi ,anti)

1. INNER JOIN

- An inner join combines tuples (rows) from two relations (tables) only when they satisfy a given condition.
- If there is no match, the tuple is not included in the result.

1.1 Theta join

- It is the most general form of join in relational algebra.
- It combines tuples from two relations based on any condition θ (not just equality).
- The condition (θ) can use operators like:
 - $=, <, >, \leq, \geq, \neq$

$$R \bowtie_{\theta} S = \sigma_{\theta}(R \times S)$$

Note: Theta join does not remove duplicate columns.

Example 1:

STUDENT		
RollNo	Name	Dept
1	Raj	D1

DEPARTMENT	
Dept	DeptName
D1	CSE

STUDENT ⋈ _{STUDENT.Dept = DEPARTMENT.Dept} DEPARTMENT				
RollNo	Name	STUDENT.Dept	DEPARTMENT.Dept	DeptName
1	Raj	D1	D1	CSE

Example 2:

Relation : STUDENT		
SID	Name	Age
1	Aditi	19
2	Rohan	22
3	Meena	21

Relation: COURSE		
CID	Title	MinAge
C1	DBMS	20
C2	Algorithms	18

Find students eligible for courses

STUDENT ⋈_{STUDENT.Age ≥ COURSE.MinAge} COURSE

SID	Name	Age	CID	Title	MinAge
1	Aditi	19	C2	Algorithms	18
2	Rohan	22	C1	DBMS	20
2	Rohan	22	C2	Algorithms	18
3	Meena	21	C1	DBMS	20
3	Meena	21	C2	Algorithms	18

1.2 Equi join

- An Equi Join is a special case of Theta Join where the condition is only equality (=).
- Tuples from two relations are joined if the specified attributes are equal.
- Unlike Natural Join, it does not remove duplicate attributes.

Relation : STUDENT		
SID	Name	Dept
1	Aditi	CSE
2	Rohan	ECE
3	Meena	CSE

Relation: DEPARTMENT	
Dept	HOD
CSE	Dr. Rao
ECE	Dr. Sen

Retrieve the names of students and their corresponding department HODs.

STUDENT ⋈ **DEPARTMENT**
STUDENT.Dept=DEPARTMENT.Dept

SID	Name	Dept (from STUDENT)	Dept (from DEPARTMENT)	HOD
1	Aditi	CSE	CSE	Dr. Rao
2	Rohan	ECE	ECE	Dr. Sen
3	Meena	CSE	CSE	Dr. Rao

1.3. Natural Join $R \bowtie S$

- A Natural Join is a special case of Equi Join.
- It automatically joins on all attributes with the same name in both relations.
- It also removes duplicate attributes from the result.

Relation : STUDENT		
SID	Name	Dept
1	Aditi	CSE
2	Rohan	ECE
3	Meena	CSE

STUDENT $\bowtie$ DEPARTMENT			
SID	Name	Dept	HOD
1	Aditi	CSE	Dr. Rao
2	Rohan	ECE	Dr. Sen
3	Meena	CSE	Dr. Rao

Relation: DEPARTMENT	
Dept	HOD
CSE	Dr. Rao
ECE	Dr. Sen

2.Outer join

- An Outer Join is like an Inner Join, but it also keeps the non-matching tuples from one or both relations.
- For the missing values, it fills with NULLs.

Types of Outer Join

2.1.Left Outer Join ($\bowtie\leftarrow$)

- Keeps all tuples from the left relation, and matching ones from the right.
- If no match, fill right side with NULL.

2.2.Right Outer Join ($\bowtie\rightarrow$)

- Keeps all tuples from the right relation, and matching ones from the left.
- If no match, fill left side with NULL.

Relation : STUDENT

RollNo	Name	DeptID
1	Raj	D1
2	Priya	D2
3	Amit	D3

Relation: DEPARTMENT

DeptID	DeptName
D1	CSE
D2	ECE
D4	MECH

LEFT OUTER JOIN

STUDENT ⋈ DEPARTMENT

RollNo	Name	DeptID	DeptName
1	Raj	D1	CSE
2	Priya	D2	ECE
3	Amit	D3	NULL

RIGHT OUTER JOIN STUDENT ⋈ DEPARTMENT			
RollNo	Name	DeptID	DeptName
1	Raj	D1	CSE
2	Priya	D2	ECE
NULL	NULL	D4	MECH

Full Outer Join STUDENT ⋈ DEPARTMENT			
RollNo	Name	DeptID	DeptName
1	Raj	D1	CSE
2	Priya	D2	ECE
3	Amit	D3	NULL
NULL	NULL	D4	MECH

3.3. FULL OUTER JOIN

- Returns all rows from both tables.
- Matching rows are combined.
- Unmatched rows from either side get NULLs.

3.1.Anti join

- An anti join returns the tuples (rows) from one relation that do not have a matching tuple in another relation.

Symbol : $R \not\bowtie S$

- $R \not\bowtie S = R - (R \bowtie S)$

Student	
Sid	Name
S1	Riya
S2	Mohit
S3	Arjun

Enroll	
Sid	Course
S1	DBMS
S3	TOC

Q.Find students who are not enrolled in any course.
 $\text{Student} \not\bowtie \text{Enroll}$ or $\text{Student} - (\text{Student} \bowtie \text{Enroll})$

Sid	Name
S2	Mohit

3 other join

3.2. semi join

➤ A semi join is similar to a normal join, but it returns only the rows from the first relation (table) that have matching rows in the second relation.

It does not include

R	
A	B
1	10
2	20
3	30

S	
B	C
10	X
30	Y

R ⋈ S → Keep only those rows of R whose B matches with B in S.	
A	B
1	10
3	30

2.3. Division Operator

- Division is used when we want to find values in one relation that are associated with all values of another relation.
- Useful in "for all" type queries (e.g., "Find students who enrolled in all courses").

Table : ENROLLED	
Student	Course
Amit	DBMS
Amit	OS
Neha	DBMS
Neha	OS
Raj	DBMS

Table : COURSE_REQ	
Course	
DBMS	
OS	

Q.Find students who have enrolled in all courses in COURSE_REQ. ENROLLED $\div$ COURSE_REQ	
Student	
Amit	
Neha	

Which Relational algebra operation require the participating table to be union compatible? Give the reason.

The Relational Algebra operations that require the participating tables to be union compatible are:

- 1. UNION ($\cup$)**
- 2. INTERSECTION ($\cap$)**
- 3. SET DIFFERENCE ($-$)**

Reason:

Because these are set operations, and for them to work properly, the tables (relations) must have:

- **Same number of attributes (columns)**
- **Same data types in corresponding columns**

Question:- Suppliers(sid, sname, address) — PK: sid

Parts(pid, pname, color) — PK: pid

Catalog(sid, pid, cost) — PK: (sid, pid), FKs: sid→Suppliers, pid→Parts

Write the relational algebra for the following **AKTU 2015-16,2018-19,2021-22**

Suppliers		
sid	sname	address
1	ABC Traders	Delhi
2	XYZ Suppliers	Mumbai
3	PQR Enterprises	Kolkata

Parts		
pid	pname	color
101	Bolt	Silver
102	Nut	Red
103	Screw	Golden
104	Wheel	Red

Catalog		
sid	pid	cost
1	101	10.50
1	102	5.00
2	101	11.00
2	103	7.50
2	104	20.00
3	102	5.50
3	103	8.00

1. Find the name of suppliers who supply some red parts .

1. Select red parts:

$R_{red} \leftarrow \sigma_{color='Red'}(Parts)$

2. Join with Catalog to find suppliers who supply red parts:

$S_{red} \leftarrow Catalog \bowtie R_{red}$

3. Join with Suppliers to get supplier names:

$Result \leftarrow \pi_{sname}(Suppliers \bowtie S_{red})$

or

combined RA expression:

$\pi_{sname}(Suppliers \bowtie (Catalog \bowtie \sigma_{color='Red'}(Parts)))$

1		
pid	pname	color
102	Nut	Red
104	Wheel	Red

2				
sid	pid	cost	pname	color
1	102	5.00	Nut	Red
2	104	20.00	Wheel	Red
3	102	5.50	Nut	Red

Join with Suppliers to get names

sid	sname	address	pid	pname	color	cost
1	ABC Traders	Delhi	102	Nut	Red	5.00
2	XYZ Suppliers	Mumbai	104	Wheel	Red	20.00
3	PQR Enterprises	Kolkata	102	Nut	Red	5.50

Project only supplier names
sname
ABC Traders
XYZ Suppliers
PQR Enterprises

2. Find the sids of suppliers who supply some red or green parts

1. Select red or green parts:

$R_{red_green} \leftarrow \sigma_{color='Red' \vee color='Green'}(Parts)$

2. Join with Catalog to find suppliers who supply these parts:

$S_{suppliers} \leftarrow Catalog \bowtie R_{red_green}$

3. Project the supplier IDs (sid):

$Result \leftarrow \pi_{sid}(S_{suppliers})$

Combined RA expression:

$\pi_{sid}(Catalog \bowtie \sigma_{color='Red' \vee color='Green'}(Parts))$

sid
1
2
3

1		
pid	pname	color
102	Nut	Red
104	Wheel	Red

2.				
sid	pid	cost	pname	color
1	102	5.00	Nut	Red
2	104	20.00	Wheel	Red
3	102	5.50	Nut	Red

3. Find the sids of suppliers who supply some red part or are at 221 packer Ave

Step 1 — select red parts

$R_{red} \leftarrow \sigma_{color='Red'}(Parts)$

1		
pid	pname	color
102	Nut	Red
104	Wheel	Red

Step 2 — join Catalog with R_{red}

$Catalog \bowtie R_{red}$

Step 3 — project supplier ids from the join

$A = \pi_{sid}(Catalog \bowtie R_{red})$

2				
sid	pid	cost	pname	color
1	102	5.00	Nut	Red
2	104	20.00	Wheel	Red
3	102	5.50	Nut	Red

Step 4 — find sids at 221 packer Ave

$B = \pi_{sid}(\sigma_{address='221packerAve'}(Suppliers))$

Result=AUB

4 empty

Result
sid
1
2
3

3
sid
1
2
3

4. Find the sids of suppliers who supply every part

$P \leftarrow \pi_{pid}(Parts).$

Then suppliers who supply every part:

$Result \leftarrow \pi_{sid}(Catalog \div P)$

(Informally: select those sid that are paired with all pid in P.)

Explanation

Step 1 — list all part ids (P)

$P \leftarrow \pi_{pid}(Parts)$

pid
101
102
103
104

Step 2 — supplier → set of parts they supply (from Catalog)
Compute the set of pid each sid supplies:

For sid = 1 → {101, 102}

For sid = 2 → {101, 103, 104}

For sid = 3 → {102, 103}

SID

5. Find the sids of suppliers who supply every red part

$R \leftarrow \pi_{pid}(\sigma_{color='Red'}(Parts))$ — set of red part ids.

Then suppliers who supply every red part:

$Result \leftarrow \pi_{sid}(Catalog \div R)$

6. Find the sids of suppliers who supply every red or green part

$$\pi_{\text{sid}}(\text{Catalog}) \div \pi_{\text{pid}}(\sigma_{\text{color}='red' \vee \text{color}='green'}(\text{Parts}))$$

Question Student (RollNo, Name, Father Name, Branch)

Book (ISBN, Title, Author, Publisher)

Issue (RollNo, ISBN, Date-of-Issue)

Write the following queries relational algebra:

1. List roll number and name of all students of the branch 'CSE'.
2. Find the name of student who has issued a book published by 'ABC' publisher.
3. List title of all books and their authors issued to a student 'RAM'.
4. List title of all books issued on or before December 1, 2020. **AKTU 2021-22**

1. List roll number and name of all students of the branch 'CSE'.

$\pi_{\text{RollNo, Name}}(\sigma_{\text{Branch}='CSE'}(\text{Student}))$

2. Find the name of student who has issued a book published by 'ABC' publisher.

$\pi_{\text{Name}}(\text{Student} \bowtie \text{Issue} \bowtie \sigma_{\text{Publisher}='ABC'}(\text{Book}))$

3. List title of all books and their authors issued to a student 'RAM'.

$\pi_{\text{Title, Author}}(\sigma_{\text{Name}='RAM'}(\text{Student}) \bowtie \text{Issue} \bowtie \text{Book})$

4. List title of all books issued on or before December

$\pi_{\text{Title}}(\sigma_{\text{DateOfIssue} \leq '2020-12-01'}(\text{Issue}) \bowtie \text{Book})$

Relational Calculus in DBMS

Relational Calculus is a non-procedural query language in which we specify what we want, not how to get it (unlike relational algebra).

There are two types:

1. Tuple Relational Calculus (TRC)
2. Domain Relational Calculus (DRC)

1. Tuple Relational Calculus (TRC)

- Tuple Relational Calculus (TRC) is a declarative / non-procedural query language for relational databases. In TRC you describe the properties of the tuples you want, not the steps to obtain them.
- It uses tuple variables (each variable represents a whole row/tuple) and first-order logic (predicates, quantifiers)

EXAMPLE

$\{ t \mid t \in \text{Employee} \wedge t.\text{dept} = \text{'Sales'} \}$

EXPLANATION:

$t \in \text{Employee} \rightarrow$ means take each row (tuple) from Employee.

$t.\text{dept} = \text{'Sales'} \rightarrow$ keep only those rows where dept is *Sales*.

OUTPUT TABLE

emp_id	name	dept	salary
1	Asha	Sales	50000
3	Meena	Sales	55000

Employee Table			
emp_id	name	dept	salary
1	Asha	Sales	50000
2	Ravi	HR	40000
3	Meena	Sales	55000
4	Arun	IT	60000

Basic form / syntax OF TRC

A TRC expression has the form:

$\{ t \mid P(t) \}$

the set of all tuples t such that predicate $P(t)$ is true.

- t is a tuple variable (it ranges over tuples of some relation).
- $P(t)$ is a formula built from:
 - atomic formulas like $t \in R$ (tuple t belongs to relation R),
 - attribute comparisons: $t.a = 5$, $t.salary > s.salary$, $t.name = u.name$,
 - logical connectives: $\wedge$, $\vee$, $\neg$,
 - quantifiers over tuple variables: $\exists u$ (there exists tuple u), $\forall u$ (for all tuples u).

There are two main types of quantifiers used in Tuple Relational Calculus (TRC)

1. Existential Quantifier ($\exists$) — “*There exists*”
2. Universal Quantifier ($\forall$) — “*For all*”

1.Existential Quantifier ($\exists$)

- The Existential Quantifier ($\exists$) means “There exists at least one...”
- It is used to say that at least one row (tuple) in a table satisfies a given condition

$\exists t (t \in \text{Employee} \wedge t.\text{dept} = \text{'HR'})$ “There exists a tuple t in the Employee table where the department is ‘HR’.”

The query is TRUE because such a tuple (Ravi) exists in the Employee table.

Employee			
emp_id	name	dept	salary
1	Asha	Sales	50000
2	Ravi	HR	40000
3	Meena	Sales	55000

2. Universal Quantifier ($\forall$)

- The Universal Quantifier ($\forall$) means “For all” or “For every”
- It is used when you want a condition to be true for every row (tuple) in a table.
- $\forall t (t \in \text{Employee} \rightarrow t.\text{salary} > 30000)$
- For every tuple t in Employee, salary is greater than 30,000
- The statement is TRUE because every employee satisfies the condition.

Employee			
emp_id	name	dept	salary
1	Asha	Sales	50000
2	Ravi	HR	40000
3	Meena	Sales	55000

Question

Database schema

Employee(emp_id, name, dept, salary)

Dept(dept, location) Write all the below question in TRC

Employee			
emp_id	name	dept	salary
1	Asha	Sales	50000
2	Ravi	HR	40000
3	Meena	Sales	55000
4	Arun	IT	60000
5	Sita	HR	45000

Dept	
dept	location
Sales	Mumbai
HR	Delhi
IT	Bangalore

1: Get all employees working in 'Sales'

$\{ t \mid t \in \text{Employee} \wedge t.\text{dept} = \text{'Sales'} \}$

2: Check if there exists an employee in 'IT'

$\exists t (t \in \text{Employee} \wedge t.\text{dept} = \text{'IT'})$

3: Check if all employees earn more than 35000

$\forall t (t \in \text{Employee} \rightarrow t.\text{salary} > 35000)$

4: Get employees working in departments located in 'Delhi'

$\{ t \mid t \in \text{Employee} \wedge \exists d (d \in \text{Dept} \wedge d.\text{dept} = t.\text{dept} \wedge d.\text{location} = \text{'Delhi'}) \}$

Solution 2
and 3 :True

Solution1			
emp_id	name	dept	salary
1	Asha	Sales	50000
3	Meena	Sales	55000

Solution 4:			
emp_id	name	dept	salary
2	Ravi	HR	40000
5	Sita	HR	45000

Safe TRC Query

- A TRC query is safe if it always produces a finite set of tuples that are drawn only from the existing values in the database.

Key points of safe queries:

- Only refer to tuples that exist in the relations.
- Use quantifiers carefully so the result is finite.
- Can be translated into SQL easily.

$\{ t \mid t \in \text{Employee} \wedge t.\text{dept} = \text{'HR'} \}$

Meaning: All employees in HR.

Produces a finite table of tuples that actually exist.

Unsafe TRC Query

- A TRC query is unsafe if it can produce:
- Infinite tuples, or Tuples containing values not present in the database
- Unsafe queries are not practically usable.

$\{ t \mid t \notin \text{Employee} \}$

- **Meaning:** All tuples not in Employee.

Note:

- TRC (with safety restrictions) is equivalent in expressive power to Relational Algebra .
Anything you can express in relational algebra can be expressed in safe TRC and vice versa.
- TRC is closer to the logical foundation of SQL — SQL's SELECT-FROM-WHERE corresponds naturally to TRC expressions

2.Domain Relational Calculus (DRC)

- DRC is a non-procedural / declarative query language for databases.
- In DRC, instead of working with tuples (rows) like TRC, we work with individual columns (domains).
- You specify what values you want for each column, not how to get them.

Basic Form / Syntax

$\{ \langle x_1, x_2, \dots, x_n \rangle \mid P(x_1, x_2, \dots, x_n) \}$

$\langle x_1, x_2, \dots, x_n \rangle \rightarrow$ domain variables representing columns.

$P(x_1, x_2, \dots, x_n) \rightarrow$ a condition (predicate) these values must satisfy

Question

Database schema

Employee(emp_id, name, dept, salary)

Write the below question in DRC

Employee			
emp_id	name	dept	salary
1	Asha	Sales	50000
2	Ravi	HR	40000
3	Meena	Sales	55000
4	Arun	IT	60000
5	Sita	HR	45000

Questions

1: Get names of employees in 'Sales'

$\{ \langle n \rangle \mid \exists e \exists d \exists s (\text{Employee}(e, n, d, s) \wedge d = \text{'Sales'}) \}$

Explanation:

- $\langle n \rangle \rightarrow$ we want only the names
- $\text{Employee}(e, n, d, s) \rightarrow e, n, d, s$ are domain variables for the columns of Employee
- $d = \text{'Sales'} \rightarrow$ department should be Sales
- $\exists e \exists d \exists s \rightarrow$ there exist values for emp_id, dept, salary

name
Asha
Meena

Questions

Employee(emp_id, name, dept, salary)

2: Get (name, salary) of employees in 'HR'

{ <n, s> | $\exists e \exists d (Employee(e, n, d, s) \wedge d = 'HR')$ }

name	salary
Ravi	40000
name	salary

SQL

SQL (Structured Query Language) is a standard language used to store, manage, and retrieve data from a relational database.

Major Categories of SQL Commands

Category	Full Form	Purpose	Examples
1.DDL	Data Definition Language	Defines and modifies database structure	CREATE, ALTER, DROP, TRUNCATE
2.DML	Data Manipulation Language	Manipulates data in tables	INSERT, UPDATE, DELETE, SELECT
3.DCL	Data Control Language	Controls access and permissions	GRANT, REVOKE
4.TCL	Transaction Control Language	Manages database transactions	COMMIT, ROLLBACK, SAVEPOINT
5.DQL	Data Query Language	Retrieves data from the database	SELECT

1 Data definition language

1.1. CREATE

CREATE command is a DDL (Data Definition Language) command used to create a new database or table in SQL

Create a Database

CREATE DATABASE SchoolDB; **OUTPUT** Database created successfully.

Create a Table

CREATE TABLE Students (

RollNo INT,

Name VARCHAR(50),

Age INT,

Marks FLOAT

);

OUTPUT

Table created successfully

Column Name	Data Type	Description
RollNo	INT	Student Roll Number
Name	VARCHAR(50)	Student Name
Age	INT	Student Age
Marks	FLOAT	Student Marks

DESC students

Field	Type	Null	Key	Default	Extra
RollNo	int	YES		NULL	
Name	varchar(50)	YES		NULL	
Age	int	YES		NULL	
Marks	float	YES		NULL	

Creating Tables with All Keys

Create Department table

```
CREATE TABLE Department (  
    DeptID INT PRIMARY KEY,      -- Primary Key  
    DeptName VARCHAR(50) UNIQUE -- Unique Key (no two departments can have same name)  
);
```

Create Employee table

CREATE TABLE Employee (

EmpID INT, -- Part of Composite Key

DeptID INT, -- Foreign Key references Department

EmpEmail VARCHAR(100) UNIQUE, -- Unique Key

EmpName VARCHAR(50),

Salary FLOAT,

PRIMARY KEY (EmpID, DeptID), -- Composite Key (EmpID + DeptID)

FOREIGN KEY (DeptID) REFERENCES Department(DeptID) -- Foreign Key

);

1.2.ALTER

ALTER command in SQL is used to modify the structure of an existing table .

You use ALTER TABLE when you want to:

- **Add a new column**
- **Delete an existing column**
- **Change the data type or size of a column**
- **Rename a column or the table**
- **Add or remove constraints (like PRIMARY KEY, UNIQUE, etc.)**

Syntax:

ALTER TABLE table_name

ADD | DROP | MODIFY | ALTER | RENAME ...

a. Add a New Column

ALTER TABLE Student

ADD age INT;

This adds a new column named age of type INT to the Student table.

b. Drop (Delete) a Column

ALTER TABLE Student

DROP COLUMN age;

This removes the age column permanently from the table.

c. Modify Data Type or Size

ALTER TABLE Student

MODIFY COLUMN name VARCHAR(100);

Changes the name column's size to 100 characters.

d. Rename a Column

ALTER TABLE Student

RENAME COLUMN name TO student_name;

e. Rename the Table

ALTER TABLE Student

RENAME TO CollegeStudent;

f. Add a Constraint

ALTER TABLE Student

ADD CONSTRAINT unique_roll UNIQUE (roll_no);

Note: it only changes structure, not data

1.3.DROP

The DROP command in SQL is used to delete an existing database object permanently — such as a table, database, view, or index.

SYNTAX:

DROP TABLE table_name;

a) Drop a Table

DROP TABLE Student;

b) Drop a Database

DROP DATABASE CollegeDB;

c) Drop a View

DROP VIEW student_view;

1.4. Truncate

- The TRUNCATE command is used to remove all rows from a table — but keep the table structure (its columns, schema, and constraints remain).

Syntax: TRUNCATE TABLE table_name;

Example:

TRUNCATE TABLE Student;

The table is now empty, but still exists

student	
id	name
1	Asha
2	Ravi
3	Meera

OUTPUT	
id	name

2. DML

It is used to manage and manipulate the data.

2.1. INSERT:

The INSERT command in SQL is a DML (Data Manipulation Language) command used to add new records (rows) into a table.

Syntax:

INSERT INTO table_name (column1, column2, column3, ...)

VALUES (value1, value2, value3, ...);

Example 1:

INSERT INTO Student (RollNo, Name, Course)

VALUES (1, 'Amit', 'CSE');

Student		
RollNo	Name	Course
<i>(empty initially)</i>		

Students		
RollNo	Name	Course
1	Amit	CSE

Example 2:

```
INSERT INTO Student  
VALUES (2, 'Riya', 'ECE');
```

Students		
RollNo	Name	Course
1	Amit	CSE
2	Riya	ECE

2.2.UPDATE

It is used to modify existing records in a table.

Syntax:

```
UPDATE table_name
```

```
SET column1 = value1, column2 = value2, ...
```

```
WHERE condition;
```

Example 1:

UPDATE Student

SET Course = 'IT'

WHERE RollNo = 2;

Student		
RollNo	Name	Course
1	Amit	CSE
2	Riya	ECE

Student(After update)

RollNo	Name	Course
1	Amit	CSE
2	Riya	IT

2.3.DELETE

It is used to remove existing records from a table.

Syntax:

DELETE FROM table_name

WHERE condition;

Example 1:

DELETE FROM Student

WHERE RollNo = 1;

Student		
RollNo	Name	Course
1	Amit	CSE
2	Riya	IT

Student		
RollNo	Name	Course
2	Riya	IT

2.4.SELECT

It is used to retrieve (view) data from one or more tables.

Syntax:

SELECT column1, column2, ...

FROM table_name

WHERE condition;

Example 1 – Select all columns

SELECT * FROM Student;

Example 2 – Select specific columns

SELECT Name, Course FROM Student;

Example 3 – With condition

SELECT * FROM Student WHERE Course = 'IT';

ORIGINAL TABLE		
Student		
RollNo	Name	Course
2	Riya	IT
3	Amit	CSE

1 Output		
RollNo	Name	Course
2	Riya	IT
3	Amit	CSE

2 output	
Name	Course
Riya	IT
Amit	CSE

3 output		
RollNo	Name	Course
2	Riya	IT

- It is used to control access and permissions to data in the database.
- It helps manage who can access or modify the data.

3.1. GRANT

It is used to give permissions to users so they can access or perform actions on database objects like tables, views, or procedures.

Syntax:

GRANT privilege_name

ON object_name

TO user_name;

Example 1:

GRANT SELECT, INSERT

ON Student

TO user1;

3.2. REVOKE

It is used to take back permissions that were previously given to a user using the GRANT command

Syntax:

REVOKE privilege_name

ON object_name

FROM user_name;

Example 1:

REVOKE INSERT

ON Student

FROM user1;

4.TCL

4.1. **COMMIT** : Saves all changes permanently

4.2. **ROLLBACK**: Cancels recent changes (undo)

4.3. **SAVEPOINT**: Sets a point to roll back to later

Example 1:

BEGIN TRANSACTION; **--starts a transaction**

UPDATE Student SET Course = 'IT' WHERE RollNo = 1; **--changes data**

SAVEPOINT A; **--marks a spot to return to if needed.**

DELETE FROM Student WHERE RollNo = 2; **--**

ROLLBACK TO A; **--undoes everything after point A (deletes undone)**

COMMIT; **--saves the remaining changes permanently.**

Aggregate Functions

- Aggregate functions in SQL are used to perform calculations on a group of values and return a single result.
- They are mostly used with the GROUP BY clause to group rows having the same values in specified columns.

Main aggregate function		
Function	Description	Example Use
1.COUNT()	Counts number of rows	Count students
2.SUM()	Adds up all values	Total marks
3.AVG()	Finds average value	Average salary
4.MAX()	Finds highest value	Highest marks
5.MIN()	Finds lowest value	Lowest marks

Table: Student		
RollNo	Name	Marks
101	Ram	80
102	Sita	90
103	Ravi	NULL
104	Riya	70

1.Count() : It is used to count the number of rows in a table or the number of non-NULL values in a column

Syntax: SELECT COUNT(column_name) FROM table_name;

Example1: SELECT COUNT(Marks) AS Total_Marks_Entries FROM Student;

Example 2: SELECT COUNT(*) FROM Student;

Output → 4 (counts all rows, including NULLs).

Total_Marks_Entries
3

2. SUM(): returns the total sum of all numeric values in a column.

syntax: SELECT SUM(column_name) FROM table_name;

Example 1: SELECT SUM(Marks) AS TotalMarks FROM Student;

Output 240(add only non-null values)

3.AVG() : It calculates the average (mean) of numeric values in a column.

Syntax: SELECT AVG(column_name) FROM table_name;

Example 1: SELECT AVG(Marks) AS AverageMarks FROM Student;

Output: 80.0 (null values are ignored.)

4. Max(): It is used find the maximum value from numeric or text column.

Syntax: SELECT MAX(column_name) FROM table_name;

Example 1: SELECT MAX(Marks) AS HighestMarks FROM Student;

Output:

HighestMarks
90

5. Min (): It is used find the minimum value from numeric or text column.

Syntax: SELECT Min(column_name) FROM table_name;

Example 1: SELECT MIN(Marks) AS LowestMarks FROM Student;

Output:

LowestMarks
70

GROUP BY CLIASE:

Employee Table		
Dept	Name	Salary
HR	Ram	40000
HR	Riya	50000
IT	Ravi	60000
IT	Sita	70000

Example :

```
SELECT Dept, COUNT(*) AS TotalEmp, AVG(Salary) AS AvgSalary
```

```
FROM Employee
```

```
GROUP BY Dept;
```

Note:

➤ GROUP BY is used to group rows that have the same values in one or more columns.

It is often used with aggregate functions (like COUNT(), SUM(), AVG(), MAX(), MIN()).

Dept	TotalEmp	AvgSalary
HR	2	45000
IT	2	65000

ORDER BY CLAUSE

It is used to sort the result in ascending (ASC) or descending (DESC) order.

Syntax : SELECT column1, column2

FROM table_name

ORDER BY column_name [ASC | DESC];

Example 1:

SELECT Name, Branch, Marks

FROM Student

ORDER BY Marks DESC;

Student			
RollNo	Name	Branch	Marks
101	Ram	CSE	80
102	Sita	CSE	90
103	Ravi	ECE	75
104	Riya	CSE	95
105	Mohit	ECE	85

OUTPUT

Name	Branch	Marks
Riya	CSE	95
Sita	CSE	90
Mohit	ECE	85
Ram	CSE	80
Ravi	ECE	75

Example Group by and Oder by

SELECT Branch, COUNT(*) AS Total_Students, AVG(Marks) AS Avg_Marks

FROM Student

GROUP BY Branch

ORDER BY Avg_Marks DESC;

OUTPUT		
Branch	Total_Students	Avg_Marks
CSE	3	88.33
ECE	2	80.00

Student			
RollNo	Name	Branch	Marks
101	Ram	CSE	80
102	Sita	CSE	90
103	Ravi	ECE	75
104	Riya	CSE	95
105	Mohit	ECE	85

Joins

Table A	
A_id	Name
1	Raj
2	Priya

Table B	
B_id	Course
1	CSE
2	ECE

1.Inner join

- It returns only those rows that have matching values in both tables.
- If there is no match, that row is not included in the result.

Example 1:

```
SELECT A.Name, B.Course  
FROM TableA  
INNER JOIN TableB  
ON A.A_id = B.B_id;
```

Name	Course
Raj	CSE
Priya	ECE



Example 2:

```
SELECT *  
FROM TableA  
INNER JOIN TableB  
ON A.A_id = B.B_id;
```

A_id	Name	B_id	Course
1	Raj	1	CSE
2	Priya	2	ECE



Table 1: Student

RollNo	Name	Branch
101	Ram	CSE
102	Sita	ECE
103	Ravi	CSE
104	Riya	ME

Table 2: Issue

RollNo	BookID	DateOfIssue
101	B1	2024-09-01
101	B2	2024-09-05
102	B3	2024-09-10

It joins tables based on equality (=) between specified columns.

Example 1:

```
SELECT Student.RollNo, Student.Name, Issue.BookID
FROM Student, Issue
WHERE Student.RollNo = Issue.RollNo;
```

RollNo	Name	BookID
101	Ram	B1
101	Ram	B2
102	Sita	B2

Example 2:

```
SELECT *
FROM Student INNERJOIN Issue
WHERE Student.RollNo = Issue.RollNo;
```

RollNo	Name	Branch	RollNo	BookID	DateOfIssue
101	Ram	CSE	101	B1	2024-09-01
101	Ram	CSE	101	B2	2024-09-05
102	Sita	ECE	102	B3	2024-09-10

Table 1: Student

RollNo	Name	Branch
101	Ram	CSE
102	Sita	ECE
103	Ravi	CSE
104	Riya	ME

Table 2: Issue

RollNo	BookID	DateOfIssue
101	B1	2024-09-01
101	B2	2024-09-05
102	B3	2024-09-10

1.2. Natural join

Automatically joins tables based on all columns with the same name and same data type.

- No need to specify the condition.

Example 1:

SELECT RollNo, Name, BookID

FROM Student

NATURAL JOIN Issue;

Student		
RollNo	Name	BookID
101	Ram	B1
101	Ram	B2
102	Sita	B2

Example 2:

SELECT *

FROM Student

NATURAL JOIN Issue;

Issue				
RollNo	Name	Branch	BookID	DateOfIssue
101	Ram	CSE	B1	2024-09-01
101	Ram	CSE	B2	2024-09-05
102	Sita	ECE	B3	2024-09-10

A table is joined with itself — used to find relationships within the same table.

EmpID(E2)	EmpName	ManagerID
1	Raj	3
2	Priya	3
3	Aman	NULL

Employee(E1)		
EmpID	Name	ManagerID
1	Raj	3
2	Priya	3
3	Aman	NULL

Example 1:

```
SELECT E1.Name AS Employee, E2.Name AS Manager
FROM Employee E1
JOIN Employee E2
ON E1.ManagerID = E2.EmpID;
```

OUTPUT	
Employee	Manager
Raj	Aman
Priya	Aman
Aman	NULL

2.Outer join

It is a type of SQL join that returns all matching rows from two tables plus the non-matching rows from one or both tables.

- It helps you see records that do not have matching data in the other table.

Table 1: Student		
RollNo	Name	Branch
101	Ram	CSE
102	Sita	ECE
103	Ravi	CSE

Table 2: Issue		
RollNo	BookID	DateOfIssue
101	B1	2024-09-01
104	B2	2024-09-05

2.1. Left outer join:

Example 1:

```
SELECT Student.RollNo, Name, BookID
FROM Student
LEFT JOIN Issue
ON Student.RollNo = Issue.RollNo;
```

RollNo	Name	BookID
101	Ram	B1
102	Sita	NULL
103	Ravi	NULL

Example 2:

```
SELECT *
FROM Student
LEFT OUTER JOIN Issue
ON Student.RollNo = Issue.RollNo;
```

RollNo	Name	Branch	RollNo	BookID	DateOfIssue
101	Ram	CSE	101	B1	2024-09-01
102	Sita	ECE	NULL	NULL	NULL
103	Ravi	ME	NULL	NULL	NULL

2.2. Right outer join:

Example 1:

```
SELECT Student.RollNo, Student.Name, Issue.BookID  
FROM Student
```

RIGHT JOIN Issue

ON Student.RollNo = Issue.RollNo;

RollNo	Name	BookID
101	Ram	B1
NULL	NULL	B2

Example 2:

```
SELECT *
```

```
FROM Student
```

```
RIGHT OUTER JOIN Issue
```

```
ON Student.RollNo = Issue.RollNo;
```

RollNo (Student)	Name	Branch	RollNo (Issue)	BookID	DateOf Issue
101	Ram	CSE	101	B1	2024-09-01
NULL	NULL	NULL	104	B2	2024-09-05

Example 1:

```
SELECT Student.RollNo, Student.Name, Issue.BookID
FROM Student
FULL JOIN Issue
ON Student.RollNo = Issue.RollNo;
```

RollNo	Name	BookID
101	Ram	B1
102	Sita	NULL
103	Ravi	NULL
104	NULL	B2

Example 2:

```
SELECT *
FROM Student
FULL OUTER JOIN Issue
ON Student.RollNo = Issue.RollNo;
```

RollNo (Student)	Name	Branch	RollNo (Issue)	BookID	DateOf Issue
101	Ram	CSE	101	B1	2024-09-01
102	Sita	ECE	NULL	NULL	NULL
103	Ravi	ME	NULL	NULL	NULL
NULL	NULL	NULL	104	B2	2024-09-05

Types of outer join

Type	Description	Includes
LEFT OUTER JOIN	Returns all rows from the left table and matched rows from the right table	Non-matching rows from left
RIGHT OUTER JOIN	Returns all rows from the right table and matched rows from the left table	Non-matching rows from right
FULL OUTER JOIN	Returns all rows when there is a match in either table	Non-matching rows from both

Trigger

- A Trigger is a stored program that is automatically executed (fired) when a specific event occurs in a table.
- It's like an automatic reaction to something that happens in your database.

Example of events that can fire a trigger:

INSERT : when a new record is added

UPDATE : when a record is modified

DELETE : when a record is deleted

Why use Triggers?

- To maintain data integrity automatically
- To log changes (audit)
- To enforce business rules (e.g., prevent certain updates or deletions)

Syntax of Trigger

CREATE OR REPLACE TRIGGER trigger_name

BEFORE | AFTER | INSTEAD OF

INSERT | UPDATE | DELETE

ON table_name

FOR EACH ROW

BEGIN

-- Trigger logic (PL/SQL code)

END;

/

Example

Whenever someone tries to reduce an employee's salary, the trigger will stop the update and show an error message.

Step 1: Create a sample table

```
CREATE TABLE Employee (  
    EmpID NUMBER PRIMARY KEY,  
    Name VARCHAR2(50),  
    Salary NUMBER  
);
```

Insert some records:

```
INSERT INTO Employee VALUES (101, 'Amit', 40000);
```

```
INSERT INTO Employee VALUES (102, 'Priya', 50000);
```

```
INSERT INTO Employee VALUES (103, 'Rohit', 60000);
```

Employee		
EmpID	Name	Salary
101	Amit	40000
102	Priya	50000
103	Rohit	60000

Step 2: Create the Trigger

CREATE OR REPLACE TRIGGER prevent_salary_decrease

BEFORE UPDATE OF Salary

ON Employee

FOR EACH ROW

BEGIN

IF :NEW.Salary < :OLD.Salary THEN

RAISE_APPLICATION_ERROR(-20001, 'Salary cannot be decreased!');

END IF;

END;

Step 3: Test the Trigger

Case 1 — Increase Salary (Allowed)

UPDATE Employee

SET Salary = 45000

WHERE EmpID = 101;

Output: 1 row updated successfully.

Case 2 — Decrease Salary (Not Allowed)

UPDATE Employee

SET Salary = 30000

WHERE EmpID = 101;

Output:

ORA-20001: Salary cannot be decreased!

ORA-06512: at "PREVENT_SALARY_DECREASE", line 3

Types of trigger

sample table:

```
CREATE TABLE Employee (  
    EmpID INT PRIMARY KEY,  
    Name VARCHAR(30),  
    Salary NUMBER  
);
```

Employee		
EmpID	Name	Salary

1. BEFORE INSERT Trigger

Fires before inserting a new record. Used to check or modify values

Example 1:

CREATE OR REPLACE TRIGGER before_insert_trigger

BEFORE INSERT ON Employee

FOR EACH ROW

BEGIN

IF :NEW.Salary < 10000 THEN

:NEW.Salary := 10000; -- set minimum salary

END IF;

END;

Explanation: If a new employee's salary is less than 10000, it will automatically change to 10000.

2. AFTER INSERT Trigger

Fires after a new record is inserted. Used for logging or auditing.

Example 1 :CREATE TABLE Emp_Log (

EmpID INT,

Action VARCHAR(20),

Action_Date DATE

);

CREATE OR REPLACE TRIGGER after_insert_trigger

AFTER INSERT ON Employee

FOR EACH ROW

BEGIN

INSERT INTO Emp_Log VALUES(:NEW.EmpID, 'Inserted', SYSDATE);

END;

Emp_Log		
EmpID	Action	Action_Date

Explanation: After an employee is inserted, a log is added to the Emp_Log table.

3. BEFORE UPDATE Trigger

Fires before updating a record. Used for validation.

Example 1: CREATE OR REPLACE TRIGGER before_update_trigger

BEFORE UPDATE ON Employee

FOR EACH ROW

BEGIN

IF :NEW.Salary < 5000 THEN

RAISE_APPLICATION_ERROR(-20002, 'Salary cannot be less than 5000');

END IF;

END;

Explanation: Prevents salary from being updated to less than 5000.

4. AFTER UPDATE Trigger

Fires after updating a record. Used to track changes

Example 1: CREATE OR REPLACE TRIGGER after_update_trigger

AFTER UPDATE ON Employee

FOR EACH ROW

BEGIN

INSERT INTO Emp_Log VALUES(:NEW.EmpID, 'Updated', SYSDATE);

END;

Explanation: When salary or name is updated, a log entry is made in Emp_Log.

5. BEFORE DELETE Trigger

➤ Fires before deleting a record. Used to back up data.

```
Example 1:CREATE TABLE Emp_Backup AS SELECT * FROM Employee WHERE 1=0;  
CREATE OR REPLACE TRIGGER before_delete_trigger  
BEFORE DELETE ON Employee  
FOR EACH ROW  
BEGIN  
    INSERT INTO Emp_Backup VALUES(:OLD.EmpID, :OLD.Name, :OLD.Salary);  
END;
```

Explanation: Before an employee is deleted, their data is copied to a backup

6. AFTER DELETE Trigger

Fires after deleting a record. Used to log the delete action

Example 1:

```
CREATE OR REPLACE TRIGGER after_delete_trigger
```

```
AFTER DELETE ON Employee
```

```
FOR EACH ROW
```

```
BEGIN
```

```
    INSERT INTO Emp_Log VALUES(:OLD.EmpID, 'Deleted', SYSDATE);
```

```
END;
```

View

- A view is a virtual table that shows data from one or more real tables.
- It does not store data permanently .
- It just shows the result of a SELECT query.

Syntax:

CREATE VIEW view_name AS

SELECT columns

FROM table_name

WHERE condition;

Employee			
EmpID	Name	Department	Salary
1	Riya	HR	40000
2	Amit	IT	60000
3	Neha	Sales	50000
4	Raj	IT	70000

Step 1: Create the base table

CREATE TABLE Employee (

EmpID INT PRIMARY KEY,

Name VARCHAR(30),

Department VARCHAR(20),

Salary INT

);

INSERT INTO Employee VALUES (1, 'Riya', 'HR', 40000);

INSERT INTO Employee VALUES (2, 'Amit', 'IT', 60000);

INSERT INTO Employee VALUES (3, 'Neha', 'Sales', 50000);

INSERT INTO Employee VALUES (4, 'Raj', 'IT', 70000);

Step 2: Create a View

Let's create a view that shows only IT department employees:

```
CREATE VIEW IT_Employees AS
```

```
SELECT Name, Salary
```

```
FROM Employee
```

```
WHERE Department = 'IT';
```

Step 3: Use the View

```
SELECT * FROM IT_Employees;
```

Step 4: Update data through View (if allowed)

```
UPDATE IT_Employees
```

```
SET Salary = 75000
```

```
WHERE Name = 'Raj';
```

Now if you check the main table Employee, Raj's salary will be updated there too.

IT_Employees	
Name	Salary
Amit	60000
Raj	70000

Step 5: Delete the View

DROP VIEW IT_Employees;

Note:

Updation Allowed (Simple View)

A simple view is made from one table and does not use:

GROUP BY

DISTINCT

Aggregate functions (like SUM, AVG)

JOIN with other tables

Then we can update, insert, or delete data through it.

Index

- It is a data structure (usually a B-tree) created on one or more columns of a table to speed up searching, sorting, and filtering.

Why use Index?

1. Without an index:

The database checks each row one by one (called *full table scan*).

2. With an index:

The database can jump directly to the required rows — just like using the page number from a book index.

CREATE TABLE Student (

RollNo INT PRIMARY KEY,

Name VARCHAR(50),

City VARCHAR(50)

);

And the table has 10,000 students. Now if you run:

SELECT * FROM Student WHERE City = 'Delhi';

- **Without an index, SQL checks all 10,000 rows.**
- **With an index on City, SQL directly jumps to all rows where City = 'Delhi'.**

How to Create an Index

1. CREATE INDEX

EXAMPLE : CREATE INDEX idx_city
ON Student (City);

- Now searching by City will be faster.

2. UNIQUE INDEX

- Used to make sure no duplicate values in a column.

EXAMPLE: CREATE UNIQUE INDEX idx_rollno
ON Student (RollNo);

- Usually created automatically when you define a PRIMARY KEY

3. COMPOSITE INDEX

Index on **multiple** columns.

EXAMPLE : CREATE INDEX idx_name_city

ON Student (Name, City);

This helps when you search using both columns:

SELECT * FROM Student WHERE Name='Riya' AND City='Delhi';

4.DROP INDEX

To remove an index

EXAMPLE: DROP INDEX idx_city;

TYPES OF INDEX

1.Clustered index

- A Clustered Index in SQL means that the table data is physically stored (sorted) in the same order as the index.
- It defines how rows are arranged inside the table.

- Data is stored and sorted based on the indexed column.
- Primary Key automatically creates a clustered index (unless specified otherwise).
- Helps in fast searching and range queries.

Example:

```
CREATE CLUSTERED INDEX idx_empid
ON Employee(EmpID);
```

Now, the table will rearrange data physically

like :

Employee		
EmpID	Name	Salary
103	Raj	25000
101	Amit	30000
102	Neha	28000

Employee		
EmpID	Name	Salary
101	Amit	30000
102	Neha	28000
103	Raj	25000

```
SELECT * FROM Employee
```

```
WHERE EmpID = 102;
```

This query will be very fast because data is already sorted by EmpID

2. Non-Clustered Index

- A Non-Clustered Index in SQL creates a separate structure (like a book's index) that stores the index key and a pointer (address) to the actual data in the table.
- It does not change the physical order of the table data.

You can create many non-clustered indexes on one table.

Employee		
EmpID	Name	Salary
103	Raj	25000
101	Amit	30000
102	Neha	28000

we create a non-clustered index on the Name column:

CREATE NONCLUSTERED INDEX idx_empname

ON Employee(Name);

How it Works:

- SQL creates a separate index table like this:
- This index helps SQL quickly find the address of the matching record
- and then go to the main table to fetch full data.

Name	Pointer (Row Address)
Amit	→ Row of Amit(2)
Neha	→ Row of Neha(3)
Raj	→ Row of Raj(1)

3. Unique:

- A Unique Index ensures that no two rows in a table have the same value in the indexed column.

It helps maintain data uniqueness , just like a rule that says “no duplicates allowed.”

- Now, you want to make sure that
- no two employees can have the same
- email address.

- You can create a Unique Index like this:

CREATE UNIQUE INDEX idx_unique_email

ON Employee(Email)

Employeee		
EmpID	Name	Email
101	Amit	amit@gmail.com
102	Neha	neha@gmail.com
103	Raj	raj@gmail.com

If you try to insert another employee with the same email:

```
INSERT INTO Employee VALUES (104, 'Ravi', 'amit@gmail.com');
```

SQL will show an error, because the email already exists.

4. Composite index

- A Composite Index is an index created on two or more columns together.
- It helps speed up queries that use multiple columns in the WHERE clause.
- Now, we create a composite index on Name and Department:

```
CREATE INDEX idx_name_dept
```

```
ON Employee(Name, Department);
```

Employee			
EmpID	Name	Department	Salary
101	Amit	HR	30000
102	Neha	IT	40000
103	Raj	HR	25000

How It Works:

SQL stores combined values like:

when you run this query :

```
SELECT * FROM Employee
```

```
WHERE Name = 'Amit' AND Department = 'HR';
```

The database uses this index to find the record quickly.

➤ Wildcards are special symbols used with the LIKE operator in SQL to search patterns in text data.

They help find data without knowing the exact value.

1. % (Percent Sign)

- Represents zero, one, or many characters.
- Used when you want to match any number of characters

Example 1: All name start with A

```
SELECT * FROM Employee  
WHERE Name LIKE 'A%';
```

Example 2: All name end with A

```
SELECT * FROM Employee  
WHERE Name LIKE '%A';
```

Example 3: Finds all names that contain 'a' anywhere

```
SELECT * FROM Employee
```

```
WHERE Name LIKE '%a%';
```

2. _ (Underscore)

➤ Represents exactly one character.

Example 1: Finds names where 'a' is the second letter.

```
SELECT * FROM Employee
```

```
WHERE Name LIKE '_a%';
```

Example 2: Finds names starting with 'A' and having total 3 letters

```
SELECT * FROM Employee
```

```
WHERE Name LIKE 'A__';
```

3. [] (Square Brackets)

Matches any one character inside the brackets.

Example 1: Finds names starting with A or R,

```
SELECT * FROM Employee
```

```
WHERE Name LIKE '[AR]%';
```

4. [^] or [!] (Not Operator in Brackets)

Matches any character not inside the brackets.

Example 1: Finds names not starting with A

```
SELECT * FROM Employee
```

```
WHERE Name LIKE '[^A]%';
```

5. - (Range inside Brackets)

Matches any character within a range.

Example 1: Finds names starting with A, B, or c

```
SELECT * FROM Employee
```

```
WHERE Name LIKE '[A-C]%';
```

Clause

clause in SQL is a part of a query that performs a specific task — like filtering, grouping, sorting, or setting conditions.

1. WHERE CLAUSE:

- The WHERE clause is used to filter rows in a table based on a specific condition.
- It helps to select only those records that meet the condition.

Example1:

SELECT * FROM Employee

WHERE Department = 'HR';

EmpID	Name	Department	Salary
101	Amit	HR	30000
103	Raj	HR	25000

Department			
EmpID	Name	Department	Salary
101	Amit	HR	30000
102	Neha	IT	40000
103	Raj	HR	25000
104	Meena	IT	35000

2. **ORDER BY**(ALREADY DONE IN PREVIOUS SLIDE)

3. **GROUP BY** (ALREADY DONE IN PREVIOUS SLIDE)

4. HAVING Clause

The HAVING clause is used to filter grouped data (data that is grouped using GROUP BY).

It works like a WHERE clause, but for aggregate functions such as COUNT(), SUM(), AVG(), et

- WHERE filters rows before grouping.
- HAVING filters groups after grouping.

Syntax:

SELECT column_name, aggregate_function(column_name)

FROM table_name

GROUP BY column_name

HAVING condition;

Q. Find departments having more than 1 employee

```
SELECT Department, COUNT(*) AS TotalEmployees
FROM Employee
GROUP BY Department
HAVING COUNT(*) > 1;
```

Department	TotalEmployees
HR	2
IT	2

Employee			
EmpID	Name	Department	Salary
101	Amit	HR	30000
102	Neha	IT	40000
103	Raj	HR	25000
104	Meena	IT	35000
105	Riya	Sales	20000

Q. Find departments where average salary > 30000

```
SELECT Department, AVG(Salary) AS AvgSalary
FROM Employee
GROUP BY Department
HAVING AVG(Salary) > 30000;
```

Department	AvgSalary
IT	37500
HR	27500 (excluded because less than 30000)

Difference between-		
Feature	WHERE	HAVING
Works On	Individual rows	Groups (after GROUP BY)
Used With	Any column	Aggregate functions
Example	WHERE Salary > 30000	HAVING AVG(Salary) > 30000

5. DISTINCT

The DISTINCT clause is used to remove duplicate values from the result of a SQL query.

It helps you get only unique records (no repetition).

Syntax: SELECT DISTINCT column_name

FROM table_name;

Example 1:

SELECT DISTINCT Department
FROM Employee;

OUTPUT
Department
HR
IT
Sales

Department		
EmpID	Name	Department
101	Amit	HR
102	Neha	IT
103	Raj	HR
104	Meena	IT
105	Riya	Sales

6.TOP/LIMIT

- The TOP or LIMIT clause is used to limit the number of rows returned by a SQL query.
- It helps you display only a specific number of records, such as the first 3 or top 5 rows.

A. Using LIMIT

SELECT * FROM Employee

LIMIT 3;

EmpID	Name	Department	Salary
101	Amit	HR	30000
102	Neha	IT	40000
103	Raj	HR	25000

B. Using TOP

➤ SELECT TOP 3 * FROM Employee;

Shows the first 3 rows (similar to LIMIT 3).

Department			
EmpID	Name	Department	Salary
101	Amit	HR	30000
102	Neha	IT	40000
103	Raj	HR	25000
104	Meena	IT	35000
105	Riya	Sales	20000

6.LIKE OPEARTOR

The LIKE operator is used in SQL to search for a specific pattern in a column.

It works with wildcards like % and _ to match text patterns.

Gateway Classes

The BETWEEN operator is used to select values within a range (inclusive of both start and end values).

EXAMPLE 1: SELECT * FROM Employee

WHERE Salary BETWEEN 30000 AND 45000;

EXAMPLE 2: SELECT * FROM Employee

WHERE Salary NOT BETWEEN 30000 AND 45000;

Employee		
EmpID	Name	Salary
1	Riya	25000
2	Amit	30000
3	Neha	40000
4	Raj	45000
5	Meena	50000

OUTPUT 2		
EmpID	Name	Salary
1	Riya	25000
5	Meena	50000

OUTPUT 1		
EmpID	Name	Salary
2	Amit	30000
3	Neha	40000
4	Raj	45000

The IN operator is used to match a value with a list of possible values.

EXAMPLE 1: SELECT * FROM Employee

WHERE Name IN ('Riya', 'Neha', 'Raj');

NOT IN

The NOT IN operator is used to exclude values

In the given list.

Example 2: SELECT * FROM Employee

WHERE Name NOT IN ('Riya', 'Neha');

Output 1:		
EmpID	Name	Salary
1	Riya	25000
3	Neha	40000
4	Raj	45000

Employee		
EmpID	Name	Salary
1	Riya	25000
2	Amit	30000
3	Neha	40000
4	Raj	45000
5	Meena	50000

OUTPUT 2		
EmpID	Name	Salary
2	Amit	30000
4	Raj	45000
5	Meena	50000

- Relational constraints are the restrictions imposed on the database contents and operations.
- They ensure the correctness of data in the database.

1.Domain Constraint-

- Domain constraint defines the domain or set of values for an attribute.
- It specifies that the value taken by the attribute must be the atomic value from its domain
- Here, value 'A' is not allowed since only integer values can be taken by the age attribute.

STUDENT		
STU_ID	Name	Age
S001	Akshay	20
S002	Abhishek	21
S003	Shashank	20
S004	Rahul	A

2. Tuple Uniqueness Constraint-

- Tuple Uniqueness constraint specifies that all the tuples must be necessarily unique in any relation.
- This relation does not satisfy the tuple uniqueness constraint since here all the tuples are not unique.

3. Key Constraint

Key constraint specifies that in any relation-

- All the values of primary key must be unique.
- The value of primary key must not be null.

STUDENT		
<u>STU_ID</u>	Name	Age
S001	Akshay	20
S001	Akshay	21
S003	Shashank	20
S004	Rahul	A

<u>STU_ID</u>	Name	Age
S001	Akshay	20
S001	Abhishek	21
S003	Shashank	20

4. Entity Integrity Constraint-

- Entity integrity constraint specifies that no attribute of primary key must contain a null value in any relation.
- This is because the presence of null value in the primary key violates the uniqueness property.
- This relation does not satisfy the entity integrity constraint as here the primary key contains a NULL value.

Student		
<u>TU_ID</u>	Name	Age
S001	Akshay	20
S002	Abhishek	21
S003	Shashank	20
	Rahul	20

5 . Referential Integrity Constraint

- This constraint is enforced when a foreign key references the primary key of a relation.
- It specifies that all the values taken by the foreign key must either be available in the relation of the primary key or be null.

Consider the following two relations- 'Student' and 'Department'.

Here, relation 'Student' references the relation 'Department'.



Student		
<u>TU_ID</u>	Name	Dept_no
S001	Akshay	D10
S002	Abhishek	D10
S003	Shashank	D11
S004	Rahul	D14

Department	
<u>Dept_no</u>	Dept_name
D10	ASET
D11	ALS
D12	ASFL
D13	ASHS

Note:

- The relation 'Student' does not satisfy the referential integrity constraint.
- This is because in relation 'Department', no value of primary key specifies department no. 14.
- Thus, referential integrity constraint is violated.

SQL DATA TYPES

Category	Data Type	Description	Example Value
Numeric	INT / INTEGER	Stores whole numbers (no decimals)	25
	SMALLINT	Stores smaller whole numbers	120
	BIGINT	Stores very large whole numbers	987654321
	DECIMAL(p, s) / NUMERIC(p, s)	Stores exact numbers with decimals (p = total digits, s = digits after decimal)	123.45
	FLOAT	Stores approximate decimal numbers (floating point)	45.67
	REAL	Similar to FLOAT but less precise	78.5

SQL DATA TYPES

Category	Data Type	Description (Easy Language)	Example Value
Character /String	CHAR(n)	Fixed-length string (always uses n characters)	'SQL '
	VARCHAR(n)	Variable-length string (uses only needed space)	'Database'
	TEXT	Stores long text data	'This is a long paragraph...'
Date and Time	DATE	Stores date only (YYYY-MM-DD)	'2025-11-01'
	TIME	Stores time only (HH:MM:SS)	'10:30:00'
	DATETIME	Stores date and time	'2025-11-01 10:30:00'
	TIMESTAMP	Stores date/time with automatic update	'2025-11-01 10:35:20'
	YEAR	Stores year only	2025

SQL DATA TYPES

Category	Data Type	Description (Easy Language)	Example Value
Binary	BINARY(n)	Fixed-length binary data (for files/images)	010101
	VARBINARY(n)	Variable-length binary data	10101
	BLOB	Stores large binary objects (e.g., images, videos)	<i>(image data)</i>
Miscellaneous	BOOLEAN	Stores TRUE or FALSE	TRUE
	ENUM	Stores one value from a predefined list	'Male', 'Female'
	SET	Stores multiple values from a predefined list	'Reading, Music'

Types of Cursors

➤ **Implicit Cursor**

➤ **Explicit Cursor**

1. Implicit Cursor

Oracle automatically creates this cursor whenever you run:

➤ **INSERT**

➤ **UPDATE**

➤ **DELETE**

➤ **SELECT INTO**

You do not have to declare it

Question: Student (RollNo, Name, Father Name, Branch)

Book (ISBN, Title, Author, Publisher)

Issue (RollNo, ISBN, Date-of-Issue)

Write the following queries relational algebra:

1. List roll number and name of all students of the branch 'CSE'.
2. Find the name of student who has issued a book published by 'ABC' publisher.
3. List title of all books and their authors issued to a student 'RAM'.
4. List title of all books issued on or before December 1, 2020.

Write it in SQL

1. Roll number & name of all CSE students.

```
SELECT RollNo, Name  
FROM Student  
WHERE Branch = 'CSE';
```

2. Name of students who issued a book published by 'ABC'

```
SELECT DISTINCT s.Name  
FROM Student s  
JOIN Issue i ON i.RollNo = s.RollNo  
JOIN Book b ON b.ISBN = i.ISBN  
WHERE b.Publisher = 'ABC';
```

3. Title and author of all books issued to student 'RAM'

```
SELECT DISTINCT b.Title, b.Author  
FROM Student s  
JOIN Issue i ON i.RollNo = s.RollNo  
JOIN Book b ON b.ISBN = i.ISBN  
WHERE s.Name = 'RAM';
```

4. Title of all books issued on or before Dec 1, 2020

SELECT DISTINCT b.Title

FROM Issue i

JOIN Book b ON b.ISBN = i.ISBN

WHERE i.Date_of_Issue <= '2020-12-01';

Unit-2 AKTU PYQs

Q.1.	Any numerical on relational algebra	AKTU 2020-21,2021-22,2022-23,2023-24
Q.2	What is relational algebra explain in detail	AKTU 2015-16,2017-18,2018-19
Q.3	Explain truncate ,drop and delete command	AKTU 2024-25
Q.4	Explain triggers and its types	AKTU 2024-25
Q.5	Explain join? Discuss all joins with example	AKTU 2024-25
Q.6	Explain equijoin in database.	AKTU 2021-22
Q.7	Explain the clause term in SQL	AKTU 2021-22
Q.8	Which Relational algebra operation require the participating table to be union compatible? Give the reason	AKTU 2021-22
Q.9	What is DDL and DML and also explain the feature of SQL	AKTU 2020-21
Q.10	Explain different integrity constraint.	AKTU 2020-21/18-19
Q.11	Write SQL query for different aggregate function.	AKTU 2020-21
Q.12	Explain tuple calculus and domain relation calculus	AKTU 2018-19



Gateway Classes



Empowering Learners, Transforming Futures

Thank You



GET IT ON
Google Play



Download on the
App Store

Helpline No- 7819 0058 53